EXPERIMENT 4

K-Nearest Neighbors (KNN) for Classification

AIM

To implement the K-Nearest Neighbors (KNN) algorithm on a real-world dataset and evaluate its performance using classification metrics.

THEORY

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. It classifies a data point based on the majority class of its **K nearest neighbors** in the feature space.

KNN is a **lazy learning algorithm**, meaning it does not build a model during training but stores the dataset and performs computation during prediction.

Mathematical Model

Euclidean Distance (commonly used in KNN):

$$d = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Where:

- xix_ixi, yiy_iyi = feature values
- ddd = distance between two data points

DATASET DESCRIPTION

This dataset is used for classifying tumors as **malignant or benign** based on various cell features.

Dataset Details:

- Total Instances: 569
- Total Features: 30
- Target Variable: `diagnosis` (M = Malignant, B = Benign)
- Dataset Type: Numerical + Categorical
- Domain: Healthcare

Input Features:

- radius_mean
- texture_mean
- perimeter_mean
- area_mean
- smoothness_mean
- and other cell-related features

CODE IMPLEMENTATION

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load the Dataset

```
from google.colab import files
uploaded = files.upload()
import pandas as pd
df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

Step 3: Data Understanding

```
df.info()
```

```
df.describe()
df.isnull().sum()
```

Step 4: Handle Target Variable

```
# Convert diagnosis column to numeric
df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})
```

Step 5: Feature Selection

```
X = df.drop(["diagnosis", "id"], axis=1)
y = df["diagnosis"]
```

Step 6: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

Step 7: Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 8: Train KNN Model

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
```

Step 9: Evaluation

```
from sklearn.metrics import accuracy_score,
classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n",
classification_report(y_test, y_pred))
```

Step 10: Confusion Matrix

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens")
plt.title("Confusion Matrix")
plt.show()
```

Step 11: Hyperparameter Tuning (Finding Best K)

```
import numpy as np

k_values = range(1, 21)
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    accuracies.append(accuracy_score(y_test, pred))

# Plot
plt.plot(k_values, accuracies)
plt.xlabel("K value")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy")
plt.show()

print("Best K:", k_values[np.argmax(accuracies)])
```

OUTPUT

- KNN successfully classified tumors
- Feature scaling significantly improved accuracy
- Optimal K value improved model performance

Screenshots:

- Dataset preview

Choose Files archive (5).zip

archive (5).zip(application/x-zip-compressed) - 49796 bytes, last modified: 3/24/2026 - 100% done

```
Saving archive (5).zip to archive (5).zip
```

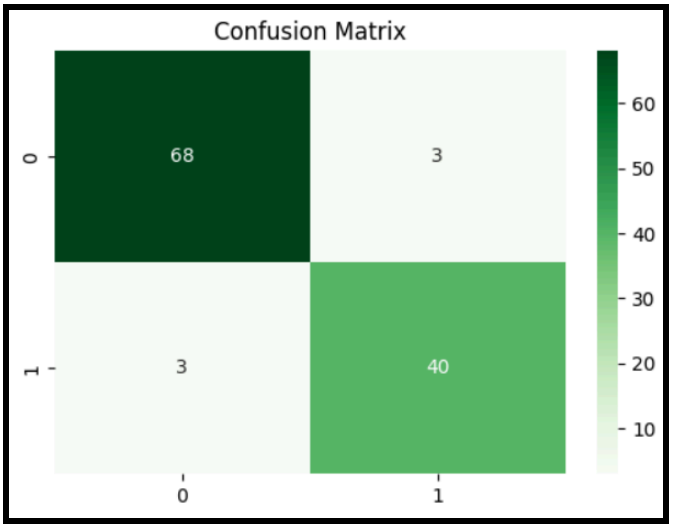
	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimete
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	

5 rows × 33 columns

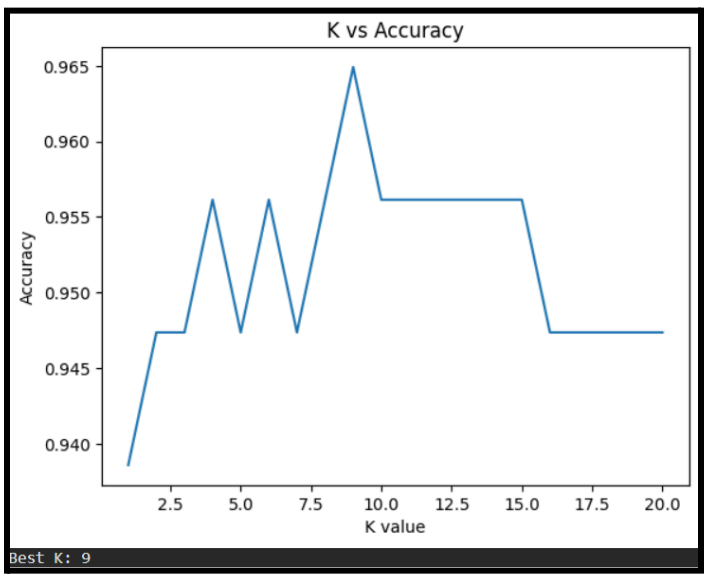
- Accuracy output

```
Accuracy: 0.9473684210526315

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96        71
           1       0.93      0.93      0.93        43

    accuracy                           0.95       114
   macro avg       0.94      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114
```

- Confusion matrix



- K vs Accuracy graph



CONCLUSION

In this experiment, the KNN algorithm was successfully implemented for classification.

The results show that:

- Proper scaling is essential
- Choosing optimal K improves performance
- KNN is simple yet effective for classification tasks